Senior Playwright + JavaScript Interview Questions & Answers

1. How would you design a scalable Playwright framework for an enterprise project?

Answer:
A scalable Playwright framework should follow layered architecture.

Key Components:
• Page Object Model (POM) for UI abstraction
• Fixtures for reusable setup/teardown
• Custom utilities for API handling, DB, logging
• Environment management using .env
• CI/CD integration
• Parallel execution
• Reporting layer
• Retry strategy
• Tag-based execution

Recommended Structure:

```
tests/
pages/
fixtures/
utils/
api/
test-data/
configs/
reports/
```

Important Design Decisions:
• Keep locators centralized
• Avoid hardcoded waits
• Reuse authentication states
• Separate business logic from test logic
• Create wrapper methods for common actions

Example:

```
test.beforeEach(async ({ loginPage }) => {
   await loginPage.login();
});
```

Senior engineers usually focus on maintainability, execution speed, debugging ease, and CI stability.

2. How does Playwright handle auto-waiting internally?

Answer:
Playwright automatically waits for elements before performing actions.

Before clicking, the element should:
• exist
• be visible
• stable
• enabled
• receive events

Example:

```
await page.locator('#submit').click();
```

Internally Playwright retries until timeout.

Why is this better than Selenium?
In Selenium:

Thread.sleep(5000)

```
In Playwright:
• no explicit waits needed mostly
• less flaky tests
• faster execution<br/><br/>

<b>Important:</b><br/>
Auto-waiting does NOT cover:
• API responses
• animations completely
• custom loaders sometimes<br/><br/>

Senior engineers combine assertions like:

await expect(loader).toBeHidden();
```

3. Difference between locator() and page.$()? Why is locator preferred?

```
<b>page.$()</b><br/>
Returns element handle immediately.

const el = await page.$('#btn');

Problems:
• stale element issues
• no retry mechanism
• not resilient<br/><br/>

<b>locator()</b><br/>
Lazy evaluation.

const button = page.locator('#btn');
await button.click();

Benefits:
• auto retry
• auto waiting
• dynamic DOM handling
• better stability<br/><br/>

Senior-level understanding:
Locators resolve elements at action time, not declaration time.
```

4. How would you reduce flaky tests in Playwright?

```
<b>Main Causes of Flakiness:</b><br/>
• hard waits
• unstable locators
• async loading
• animation timing
• shared test data
• environment instability<br/><br/>

<b>Best Practices:</b><br/>
1. Use stable locators like getByTestId()<br/>
2. Avoid waitForTimeout()<br/>
3. Isolate tests<br/>
4. Use API mocking<br/>
5. Configure retries<br/>
6. Use tracing/videos for debugging
```

5. Explain Playwright fixtures and why they are powerful.

```
<b>Answer:</b><br/>
Fixtures are reusable test context providers.<br/><br/>
<b>Example:</b>
```

```
export const test = base.extend({
   loginPage: async ({ page }, use) => {
      await use(new LoginPage(page));
   }
});
```

Benefits:
• dependency injection
• reusable setup
• cleaner tests
• parallel-safe architecture

Senior-level use cases include authenticated sessions, DB connections, API clients, browser contexts, and mock

6. How do browser contexts improve execution efficiency?

Answer:
Browser context = isolated browser session.

Instead of launching multiple browsers:

```
const context = await browser.newContext();
```

Benefits:
• faster execution
• isolated cookies/session
• parallel testing
• multi-user scenarios

7. How would you implement authentication once and reuse it across tests?

Approach:
Use storage state.

Step 1:

```
await page.context().storageState({
   path: 'auth.json'
});
```

Step 2:

```
use: {
   storageState: 'auth.json'
}
```

Benefits:
• avoids repeated login
• faster suite
• less server load

8. How do you handle network mocking in Playwright?

Example:

```
await page.route('**/api/users', route => {
   route.fulfill({
      status: 200,
      body: JSON.stringify({ name: 'Tanu' })
   });
});
```

Use cases:
• unstable APIs
• negative scenarios
• third-party dependency isolation
• performance testing

9. Difference between browser.newPage() and context.newPage()?

```
<b>Wrong Approach:</b>

const page = await browser.newPage();

Creates implicit context.<br/><br/>

<b>Preferred:</b>

const context = await browser.newContext();
const page = await context.newPage();

Why?
• better session isolation
• cookie control
• permissions handling
```

10. How does Playwright support parallel execution safely?

```
Each test gets isolated browser context.<br/><br/>

<b>Configuration:</b>

workers: 4

Challenges:
• shared test data
• race conditions
• environment conflicts
```

11. How would you integrate Playwright in CI/CD?

```
<b>Pipeline Steps:</b><br/>
• install dependencies<br/>
• install browsers<br/>
• run tests<br/>
• publish reports<br/>
• archive artifacts<br/><br/>

<b>Example GitHub Actions:</b>

- run: npm ci
- run: npx playwright install
- run: npx playwright test

Important considerations:
• headless execution
• retries
• shard execution
• artifact retention
```

12. How do you capture logs, screenshots, and traces for debugging?

```
<b>Screenshot:</b>

await page.screenshot();

<b>Trace:</b>

use: {
   trace: 'on-first-retry'
}

<b>Video:</b>
```

video: 'retain-on-failure'

```
Tracing provides DOM snapshot, network logs, console logs, and action timeline.
```

13. Explain API testing capabilities in Playwright.

```
<b>Example:</b>

const response = await request.get('/users');

Benefits:
• UI + API in same framework
• token sharing
• end-to-end validation
```

14. How do you handle dynamic locators in Playwright?

```
<b>Example:</b>
```

page.locator(`text=${username}`)

```
Better approach:

page.locator('.user-card')
    .filter({ hasText: 'Tanu' })

Why?
More stable than XPath-heavy approaches.
```

15. Explain Playwright retry mechanism.

```
<b>Configuration:</b>

retries: 2

Retries should NOT hide bad tests.<br/><br/>
Senior strategy:
• retries only in CI
• analyze flaky patterns
• quarantine unstable tests
```

16. How would you optimize Playwright execution time?

```
Techniques:
• parallel execution
• storage state reuse
• API-based setup
• reduce UI dependency
• headless mode
• selective tagging<br/><br/>

<b>Example:</b>
```

npx playwright test --grep @smoke

```
Advanced:
• shard execution
• Docker execution
• browser reuse
```

17. How do you validate downloads in Playwright?

```
<b>Example:</b>
```

```
const downloadPromise = page.waitForEvent('download');

await page.click('#download');

const download = await downloadPromise;

Validation:
```

expect(download.suggestedFilename())

```
Advanced validation includes PDF text and CSV rows.
```

18. How would you test multi-tab or popup scenarios?

```
<b>Example:</b>

const [newPage] = await Promise.all([
   context.waitForEvent('page'),
   page.click('#open')
]);

Important:
Use context-level listeners.
```

19. Explain Playwright hooks and best practices.

```
Hooks:
• beforeAll
• beforeEach
• afterEach
• afterAll<br/><br/>

Best Practices:
• keep hooks lightweight
• avoid test dependency
• cleanup properly
```

20. How would you design hybrid UI + API automation strategy?

```
Best enterprise strategy:
Use APIs for:
• setup
• cleanup
• data creation<br/><br/>

Use UI only for actual business validation.<br/><br/>

<b>Example:</b>

await api.createUser();

Benefits:
• faster tests
• less flaky
• stable environments
• reduced execution time
```